

Systems Programming 14

Jesse Parron

Bash - The Bourne Again Shell

- The Bourne Again Shell (bash) is a command interpreter and a high-level programming language.
- As a command interpreter, it processes commands you enter on the command line in response to a prompt.
- When you use the shell as a programming language, it processes commands stored in files called **shell scripts**.

BASH

- The Bourne Again Shell is based on the Bourne Shell, which was written by Steve Bourne of AT&T's Bell Laboratories.
- On UNIX systems the original Bourne Shell is named sh.
- On many Linux systems sh is a symbolic link to bash or dash, ensuring scripts that require the presence of the Bourne Shell still run. When called as sh, bash does its best to emulate the original Bourne Shell.

Writing a simple script

- A shell script is a file that holds commands the shell can execute. The commands in a shell script can be any commands you can enter in response to a shell prompt.
- For example, a command in a shell script might run a utility, a compiled program, or another shell script.

Scripting

- To execute a shell script by giving its name as a command, you must have permission to read and execute the file that contains the script.
- When you create a shell script using an editor, the file does not typically have its execute permission set.

```
$ cat whoson
date
echo "Users Currently Logged In"
who

$ ./whoson
bash: ./whoson: Permission denied
```

Scripting

- To update these permissions we use a command known as **chmod**
- Chmod changes permissions for files or directories for users
- To update our file we can run **chmod +x filename** in the terminal

Scripting

- When scripting we put a special sequence of characters on the first line of a shell script to tell the operating system which shell should execute the file.
- **#! (known as hasbang or shebang)** are the first two characters of a script, the system interprets the characters that follow as the absolute pathname of the program that is to execute the script.
 - This pathname can point to any program, not just a shell.

```
$ cat bash_script
#!/bin/bash
echo "This is a Bourne Again Shell script."
```

Scripting

- If a pound sign or hash (#) in the first character position of the first line of a script is not immediately followed by an exclamation point (!) or if a hash occurs in any other location in a script, the shell interprets it as the beginning of a comment.

Writing a script

- Next let's use **vim** or **vi** to create a script file
- For example:
 - vim my_script.sh
- **Things to note**
 - Bash scripting is case sensitive and space sensitive
 - For certain declarations and arguments, the terminal will read variables after whitespaces as a new command. So spacing is extremely important.

Writing a script

- Now let's make a script that:
- Echoes a sentence
- And lists contents of a directory

```
#!/bin/bash
```

```
echo "my name is Jesse"
```

```
ls "/home/parronj/Desktop"
```

Writing a script

- Now, since we just created this file, it probably does not have the permissions we need to execute it.
- Lets check using **ls -l "filename"**

Writing a script

- Since it does not have the permissions we need, we can update its permissions using chmod.
- Lets do **chmod +x "filename.sh"**
- + is adding a permission
- x is the execute permission

Writing a script

- There are multiple ways to update file permissions in Linux
- When using `chmod`, you can modify permissions for the user, group, and others all at once by specifying a three-digit numeric code
- Each digit represents a combination of read (4), write (2), and execute (1) permissions:
 - 777 → Read, write, and execute for the owner, group, and others.
 - 666 → Read and write for everyone
 - 444 → Read-only for everyone
- You can adjust these numbers to set specific permissions for each category (owner, group, others).

Writing a script

- You can also update a specific category manually by using r, w, x
 - r=read
 - w=write
 - x=execute
- Say we have a script named test.sh
- We can do `chmod u=rwx,g=rw,o=r test.sh`
- This will make it so the user has read, write, execute permissions
- Group has read and write
- And other can only read

Writing a Script

- Now we can execute our script
- We can type `./filename.sh` in the command line, and we should get an output from our script.
- Remember that `.` denotes our current working directory
- `./` denotes we want to run a file in our current working directory
 - o Load it into memory

Variables in Bash Scripting

- Variable names are unmarked in assignments but prefixed with a dollar sign when their values are referenced.
- For example:

```
#!/bin/bash  
my_name="Jesse"  
echo $my_name
```

- This assigns the string “Jesse” to variable my_name
- We can then access the variable using the \$

Variables in Bash Scripting

- An important thing to note is **do not put spaces** around the = symbol or the shell will mistake your variable name for a command name.
- If you wanted to output your variable in a sentence and want letters to follow it, you can use curly brackets to state where our variable starts and ends
- Example:
 - `$ echo "Saved ${rev}th version of file.txt"`
 - *Saved 8th version of file.txt*

Printing in Bash

- As we have already seen, we can simply print our variables and strings using `echo`.
- `echo` is great for printing simple outputs that don't need any formatting
- If we wanted to format our outputs, we can use `printf`
- `printf` is used when you need precise control over the output format, such as aligning text, formatting numbers, or including special characters
- Example:
 - `$ echo "\taa\tbb\tcc\n"`
 - `\taa\tbb\tcc\n`
 - `$ printf "\taa\tbb\tcc\n"`
 - `aa bb cc`

Taking input

- We can also take input from a user using the **read** command
- Here is an example of how we can take input:
 - *#!/bin/bash*
 - *echo -n "Enter your name: "*
 - *read user_name*
 - *echo "Hello \$user_name"*

Backticks and double quotes

- Back quotes, also known as backticks, are treated similarly to double quotes, but they have the additional effect of executing the contents of the string as a shell command and replacing the string with the command's output
- Example:
 - *\$ echo "There are `wc -l /etc/passwd` lines in the passwd file."*
 - *There are 28 lines in the passwd file.*

If statements

- Just like other languages, we have conditional statements like an if statement.
- The syntax for an if statement in bash is below:

#!/bin/bash

if [condition]; then

#code to be executed inside the if

fi

- fi marks the end of an if statement

Example if statement

```
#!/bin/bash  
echo -n "Enter your name: "  
read user_name  
if [ -n "$user_name" ]; then  
    echo "Hello $user_name!"  
    exit 0  
else  
    echo "You did not tell me your name!"  
    exit 1  
fi
```

If statements

- Just like other languages, if statements can also have else if, we can denote this with elif.
- Additionally, if statements can be nested.

#!/bin/bash

If [condition 1]; then

Code to be executed if condition 1 is true

elif [condition 2]; then

Code to be executed if condition 2 is true

else

code to be executed if none of the conditions prove true

fi

Functions

- Just like other languages, we can use functions to define a task or operation and later call it and use it in other parts of the script.

1. The first way:

```
#!/bin/bash  
myFunction (){  
echo "Hello world"  
}  
#function call  
myFunction
```


Loops

- Lets start with for loops, there are a few ways we can define one
- The first, we can simply just loop over a set of variables
 - *for n in a b c;*
 - *do*
 - *echo \$n*
 - *done*
- This will ultimately just output **a b c**

Loops

- We can also define a for loop using a range, we can denote this by using curly brackets and indicating a start and stop point.
 - *for n in {1..15};*
 - *do*
 - *echo \$n*
 - *done*
- This will output a range of numbers starting from 1 and ending at 15

Loops

- We can also specify how much we want to increment by in our for loop by adding an additional value in our curly brackets.
 - *for n in {1..15..2};*
 - *do*
 - *echo \$n*
 - *done*
- This will output values starting at 1 and increment by 2 each time
 - So we can get 1,3,5,7,9,11,13,15

While loop

- Additionally we can also use while loops
- Similar to for loops the syntax is:
 - *while [condition];*
 - *do*
 - *# statements*
 - *# commands*
 - *done*

While Loop

- An example of a while loop can be seen below:
 - *#!/bin/bash*
 - *file=temp.txt*
while read -r line;
do
echo \$line
done < "\$file"
- This script reads each line from the file “temp.txt” and prints it to the console until there are no more lines left in the file

Arrays

- Before seeing how we can loop an array, lets touch on how we can form an array.
- There are a few different ways.
- We can use declare -a follow by the name of our array.
- For example:
 - *declare -a students*
 - *students[0]="Jesse"*
 - *students[1]="Omar"*
 - *students[2]="Laury"*

Arrays

- If we want to declare a static array we can simply do:
 - *programmingArray=("Java" "Python" "Ruby" "Perl")*

Printing arrays

- To print an array we use the following syntax `echo "${array_name[@]}"`
- We use the `[@]` as an index to the array to display all the elements
- Example:
 - `declare -a students`
 - `students[0]="Jesse"`
 - `students[1]="Omar"`
 - `students[2]="Laury"`
 - `students[3]="Garrett"`
 - `echo "${students[@]}"`

Looping arrays

- We can also loop through an array by using a for loop as described earlier, an example can be seen below

```
declare -a students
```

```
students[0]="Jesse"
```

```
students[1]="Omar"
```

```
students[2]="Laury"
```

```
for i in ${students[@]}
```

```
do
```

```
echo -e "$i \n"
```

```
done
```

Lets try another script

- In this script we will utilize a loop, and user input
- Let's ask a user to input a value and we will calculate the factorial
- Do **vim factsc.sh**

Factorial Script

```
- #!/bin/bash
-
- factorial(){
-     number=$1
-     fact=1
-
-     for ((i = 2; i<=number; i++))
-     do
-         fact=$((fact * i))
-     done
-
-     echo $fact
- }
-
- read -p "enter a number: " num
-
- echo "Factorial of $num is: $(factorial $num)"
```

Scripting

- Lets do one more program to showcase how to use if statements in bash
- `#!/bin/bash`
-
- `read -p "Enter a number: " number`
- `if (( number > 0 )); then`
- `echo "The number is positive."`
- `elif (( number < 0 )); then`
- `echo "The number is negative."`
- `else`
- `echo "The number is zero."`
- `fi`